

K L Deemed to be University

Department of Computer Science and Engineering
2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 6: Organized Routing and CRUD Operations

Name: GAYATHRI SRIVALLI KAJULURI **Roll No:** 2520080010

Course: 25CS1302E **CO Mapping:** CO3

Session: Practical 6

Aim

To separate the BookFlow API into a public catalog router (GET /books, GET /books/{book_id}) and a librarian admin router (POST, PUT, DELETE /admin/books) with APIRouter, plug both into main.py with include_router(), and validate POST/PUT bodies with Pydantic (non-negative price, ISBN as a string).

Theory

APIRouter: FastAPI's APIRouter is used to organize related endpoints into separate route modules. In this experiment, the public catalog operations and librarian operations are kept in different routers and then registered in the main application using include_router().

CRUD operations: CRUD represents Create, Read, Update and Delete. The admin router performs Create through POST, Update through PUT and Delete through DELETE, while the catalog router provides Read operations through GET requests.

Pydantic validation: Request models validate incoming JSON data before it reaches the database. The book schema ensures that ISBN is supplied as a string and prevents a negative price using a greater-than-or-equal-to-zero constraint.

Path and query parameters: The book_id value in /books/{book_id} is a path parameter. The public list endpoint also supports search, skip and limit as query parameters.

Program

bookflow_api/main.py

```
from contextlib import
asynccontextmanager from typing import
AsyncIterator from fastapi import FastAPI
from sqlalchemy import text from database
import engine from routes import admin,
catalog

@asynccontextmanager async def lifespan(_: FastAPI) -
> AsyncIterator[None]:     async with engine.begin()
as conn:
    await conn.execute(          text("ALTER TABLE books ADD COLUMN IF NOT
EXISTS price NUMERIC(10,2)")      )          await conn.execute(text(
"ALTER TABLE books ADD COLUMN IF NOT EXISTS status VARCHAR(20) "
"NOT NULL DEFAULT
'AVAILABLE'"          ))          yield
await engine.dispose()

app = FastAPI()
```

```

        title="BookFlow Library
API",        version="2.0.0",
lifespan=lifespan )

app.include_router(catalog.router)
app.include_router(admin.router)

@app.get("/", tags=["Health"]) async def root() -> dict:
return {"service": "BookFlow Library API", "status": "running"}

```

bookflow_api/database.py

```

import os from typing import AsyncIterator from
sqlalchemy.ext.asyncio import ( AsyncSession,
async_sessionmaker, create_async_engine )
from sqlalchemy.orm import DeclarativeBase

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql+asyncpg://postgres:postgres@localhost:5432/bookflow_db"
)

engine = create_async_engine(DATABASE_URL, pool_size=10, max_overflow=20)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)

class Base(DeclarativeBase):
    pass

async def get_db() ->
AsyncIterator[AsyncSession]:        async with
SessionLocal() as session:        yield session

```

bookflow_api/models.py

```

from decimal import Decimal from typing import
Optional from sqlalchemy import Integer,
Numeric, String from sqlalchemy.orm import
Mapped, mapped_column from database import Base

class Book(Base):
    __tablename__ = "books"

    book_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(200), nullable=False)        isbn:
Mapped[str] = mapped_column(String(13), unique=True, nullable=False)
    published_year: Mapped[Optional[int]] = mapped_column(Integer)        price:
Mapped[Optional[Decimal]] = mapped_column(Numeric(10, 2))        status:
Mapped[str] = mapped_column(        String(20), nullable=False,
default="AVAILABLE"        )

```

bookflow_api/schemas.py

```

from decimal import Decimal from enum import
Enum from typing import Optional from pydantic
import BaseModel, ConfigDict, Field

class BookStatus(str, Enum):
    AVAILABLE = "AVAILABLE"
    BORROWED = "BORROWED"
    LOST = "LOST"

class BookCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)        isbn:
str = Field(min_length=10, max_length=13)        published_year:
Optional[int] = Field(default=None, lt=2027)        price: Decimal
= Field(ge=0, max_digits=10, decimal_places=2)        status:
BookStatus = BookStatus.AVAILABLE

class BookUpdate(BaseModel):
    title: Optional[str] = Field(default=None, min_length=1, max_length=200)
    isbn: Optional[str] = Field(default=None, min_length=10, max_length=13)
    published_year: Optional[int] = Field(default=None, lt=2027)
    price: Optional[Decimal] = Field(
        default=None, ge=0, max_digits=10, decimal_places=2        )
    status: Optional[BookStatus] = None

class BookOut(BaseModel):        model_config =
ConfigDict(from_attributes=True)

```

```

    book_id: int        title: str
isbn: str        published_year:
Optional[int] = None    price:
Optional[Decimal] = None    status:
str

```

bookflow_api/routes/catalog.py

```

from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from database import get_db
from models import Book
from schemas import BookOut

router = APIRouter(prefix="/books", tags=["Public Catalog"])

@router.get("", response_model=List[BookOut])
async def list_books(
    search: Optional[str] = Query(default=None),
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=20, ge=1, le=100),
    db: AsyncSession = Depends(get_db),
) -> List[Book]:
    statement = select(Book).order_by(Book.book_id).offset(skip).limit(limit)

    if search:
        statement = statement.where(Book.title.ilike(f"%{search}%"))

    return list((await db.scalars(statement)).all())

@router.get("/{book_id}", response_model=BookOut)
async def get_book(
    book_id: int,
    db: AsyncSession = Depends(get_db),
) -> Book:
    book = await db.get(Book, book_id)

    if book is None:
        raise HTTPException(
            status.HTTP_404_NOT_FOUND,
            f"Book {book_id} not found"
        )

    return book

```

bookflow_api/routes/admin.py

```

from fastapi import APIRouter, Depends, HTTPException, Response, status
from sqlalchemy.exc import IntegrityError
from sqlalchemy.ext.asyncio import AsyncSession
from database import get_db
from models import Book
from schemas import BookCreate, BookOut, BookUpdate

router = APIRouter(prefix="/admin/books", tags=["Librarian Admin"])

async def _get_or_404(
    db: AsyncSession,
    book_id: int,
) -> Book:
    book = await db.get(Book, book_id)

    if book is None:
        raise HTTPException(
            status.HTTP_404_NOT_FOUND,
            f"Book {book_id} not found"
        )

    return book

async def _commit(
    db: AsyncSession,
) -> None:
    try:
        await db.commit()
    except IntegrityError:
        await db.rollback()
        raise HTTPException(
            status.HTTP_409_CONFLICT,
            "A book with this ISBN already exists"
        )

@router.post("", response_model=BookOut, status_code=status.HTTP_201_CREATED)
async def add_book(
    payload: BookCreate,
    db: AsyncSession = Depends(get_db),
) -> Book:
    book = Book(**payload.model_dump())
    db.add(book)
    await _commit(db)
    await db.refresh(book)
    return book

```

```

@router.put("/{book_id}",
response_model=BookOut) async def update_book(
book_id: int,      payload: BookUpdate,      db:
AsyncSession = Depends(get_db) ) -> Book:
book = await _get_or_404(db, book_id)

    for field, value in payload.model_dump(exclude_unset=True).items():
setattr(book, field, value)

    await _commit(db)
await db.refresh(book)
return book

@router.delete("/{book_id}",
status_code=status.HTTP_204_NO_CONTENT) async def delete_book(
book_id: int,      db: AsyncSession = Depends(get_db) ) -> Response:
book = await _get_or_404(db, book_id)      await db.delete(book)
await db.commit()
    return Response(status_code=status.HTTP_204_NO_CONTENT)

```

test_p6.sh

```

#!/bin/bash

BASE=http://127.0.0.1:8000

echo "### Registered routes"
python3 list_routes.py

echo "### 1. Public: GET /books"
curl "$BASE/books?limit=5"

echo "### 2. Public: GET /books/{book_id}"
curl "$BASE/books/3"

echo "### 3. Missing book"
curl "$BASE/books/999"

echo "### 4. Admin: POST /admin/books" curl -X POST "$BASE/admin/books" \ -H 'Content-
Type: application/json' \ -d '{"title":"Fluent
Python","isbn":"9781492056355","published_year":2022,"price":2450.00}'

echo "### 5. Invalid price and numeric ISBN" curl -X POST
"$BASE/admin/books" \ -H 'Content-Type: application/json'
\ -d '{"title":"Bad Price","isbn":9781492056356,"price":-
10}'

echo "### 6. Admin: PUT
/admin/books/{book_id}" curl -X PUT
"$BASE/admin/books/17" \ -H 'Content-Type:
application/json' \ -d
'{"price":1999.00,"status":"BORROWED}"

echo "### 7. Check updated book"
curl "$BASE/books/17"

echo "### 8. Admin: DELETE /admin/books/{book_id}"
curl -X DELETE "$BASE/admin/books/17"
echo "### 9. Check after deletion"
curl "$BASE/books/17"

```

Output

Project structure and server start

```
$ tree
bookflow_api
bookflow_api/ |--
main.py
|-- database.py
|-- models.py
|-- schemas.py
`-- routes/
    |-- __init__.py
    |-- catalog.py
    `-- admin.py

$ uvicorn main:app --port 8000
INFO: Started server process INFO:
Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000
```

Registered routes

GET	/books	[Public Catalog]
GET	/books/{book_id}	[Public Catalog]
POST	/admin/books	[Librarian Admin]
PUT	/admin/books/{book_id}	[Librarian Admin]
DELETE	/admin/books/{book_id}	[Librarian Admin]
GET	/	[Health]

1. GET /books?limit=5

```
HTTP 200
[
  {"book_id":1,"title":"Clean Code","isbn":"9780132350884",
   "published_year":2008,"price":null,"status":"AVAILABLE"},
  {"book_id":2,"title":"Database System Concepts","isbn":"9780078022159",
   "published_year":2019,"price":null,"status":"AVAILABLE"},
  {"book_id":3,"title":"The Martian","isbn":"9780553418026",
   "published_year":2014,"price":null,"status":"AVAILABLE"},
  {"book_id":7,"title":"Dune","isbn":"9780441172719",
   "published_year":2019,"price":null,"status":"AVAILABLE"},
  {"book_id":8,"title":"Atomic Habits","isbn":"9780735211292",
   "published_year":2018,"price":null,"status":"AVAILABLE"}
]
```

2. GET /books/3

```
HTTP 200
{
  "book_id": 3,
  "title": "The Martian",
  "isbn": "9780553418026",
  "published_year": 2014,
  "price": null,
  "status": "AVAILABLE"
}
```

3. GET /books/999

```
HTTP 404
{"detail":"Book 999 not found"}
```

4. POST /admin/books

```
HTTP 201
{
  "book_id": 17,
  "title": "Fluent Python",
  "isbn": "9781492056355",
  "published_year": 2022,
  "price": "2450.00",
  "status": "AVAILABLE"
}
```

5. Invalid POST: negative price and numeric ISBN

```
HTTP 422
Validation errors:
- isbn: Input should be a valid string
- price: Input should be greater than or equal to 0
```

6. PUT /admin/books/17

```
HTTP 200
{
  "book_id": 17,
  "title": "Fluent Python",
  "isbn": "9781492056355",
  "published_year": 2022,
  "price": "1999.00",
  "status": "BORROWED"
}
```

7. GET /books/17 after update

```
HTTP 200
{
  "book_id": 17,
  "title": "Fluent Python",
  "isbn": "9781492056355",
  "published_year": 2022,
  "price": "1999.00",
  "status": "BORROWED"
}
```

8. DELETE /admin/books/17

```
HTTP 204
```

9. GET /books/17 after deletion

```
HTTP 404
{"detail": "Book 17 not found"}
```

Result / Conclusion

The BookFlow API was successfully organized into separate public catalog and librarian admin routers. The experiment demonstrated GET, POST, PUT and DELETE operations, along with path and query parameters. Pydantic validation rejected invalid input such as a numeric ISBN and a negative price. The successful status codes and final 404 response after deletion confirm that the CRUD workflow and route separation are functioning correctly.